

Microservicio de aprendizaje no supervisado

[Completar: título del proyecto / problemática de la app móvil]

Datos del equipo

Equipo	<i>[Completar: Nombre(s) del equipo]</i>
Integrantes	<i>[Completar: Integrante 1, Integrante 2, ...]</i>
Correo de contacto	<i>[Completar: correo@upchiapas.edu.mx]</i>
Modalidad	<i>[Completar: Modelo local / Modelo mediante API]</i>
Algoritmo no supervisado	<i>[Completar: k-means, DBSCAN, Isolation Forest, Autoencoder, ...]</i>
Repositorio	<i>[Completar: URL del repositorio Git]</i>

Índice

1. Introducción	3
1.1. Alcance de este entregable	3
1.2. Modalidades de implementación	3
2. Objetivos	3
2.1. Objetivo general	4
2.2. Objetivos específicos	4
2.3. Criterios de aceptación mínimos	4
3. Descripción del problema	4
3.1. Contexto del proyecto móvil	4
3.2. Necesidad analítica	4
3.3. Pregunta analítica	5
3.4. Vínculo con la app móvil (integración futura	5
4. Arquitectura del componente	5
4.1. Vista lógica	5
4.2. Stack tecnológico	5
4.3. Flujo de una inferencia	6
4.4. Integración futura con la app móvil	6
5. Modelo de aprendizaje no supervisado	6
5.1. Selección del algoritmo	6
5.2. Fundamento breve	6
5.3. Datos de entrenamiento	6
5.4. Hiperparámetros	7
5.5. Entrenamiento y artefacto	7
5.6. Métricas de evaluación (no supervisado)	7
6. Preprocesamiento y validación de entrada	7
6.1. Esquema de entrada	8
6.2. Validaciones implementadas	8
6.3. Transformaciones	8
6.4. Consistencia entrenamiento vs. producción	8
7. Implementación del microservicio	9
7.1. Repositorio	9
7.2. Arranque del servicio	9
7.3. Fragmento representativo	9
7.4. Manejo de errores	9
7.5. Dependencias principales	9
8. Documentación de la API REST	10
8.1. Base URL y versionado	10
8.2. Endpoints	10
8.3. Ejemplo de inferencia	10
8.4. Consulta de historial	10

8.5. Documentación Swagger	11
9. Base de datos e historial de inferencias	11
9.1. Motor y justificación	11
9.2. Modelo de datos	12
9.3. Operaciones	12
9.4. Diagrama entidad-relación (opcional)	12
9.5. Privacidad	12
10.Pruebas y colección de API	12
10.1. Estrategia de pruebas	13
10.2. Colección Postman / Bruno	13
10.3. Ejecución de pruebas automatizadas	13
10.4. Checklist de entrega	13
11.Resultados	14
11.1. Resultados del modelo	14
11.2. Ejemplos de inferencia	14
11.3. Desempeño del servicio	14
11.4. Visualizaciones	14
11.5. Limitaciones conocidas	15
12.Conclusiones y trabajo futuro	15
12.1. Conclusiones	15
12.2. Integración con la aplicación móvil	15
12.3. Trabajo futuro	15
12.4. Entregables finales del hito	16
A. Anexos	16
A.1. Anexo A — Especificación OpenAPI	16
A.2. Anexo B — Enlaces de entrega	16
A.3. Anexo C — Guion sugerido del video (5–10 min)	16
A.4. Anexo D — Registro de decisiones técnicas (opcional)	17

1. Introducción

Para qué sirve esta sección

Contextualiza el hito del proyecto final: un microservicio de **aprendizaje no supervisado** que funcionará de forma **independiente** antes de integrarse a la aplicación móvil.

Este reporte documenta el primer componente de inteligencia artificial del proyecto final del curso de **Minería de datos**. En esta etapa **no** se evalúa la integración con la aplicación móvil, sino el funcionamiento autónomo del microservicio y la correcta implementación del modelo no supervisado acordado en la propuesta del equipo.

El microservicio expone una *API REST* que recibe datos desde un cliente, valida y preprocesa la entrada, ejecuta el algoritmo seleccionado y devuelve resultados en JSON [1, 2]. Además, persiste cada inferencia en base de datos y ofrece un endpoint de consulta histórica. La documentación interactiva se publica con Swagger/OpenAPI [3].

1.1. Alcance de este entregable

- Código fuente del microservicio (*[Completar: URL del repositorio Git]*).
- Este documento en PDF (compilado desde L^AT_EX).
- Notebook con evidencia de entrenamiento y pruebas.
- Diagrama de arquitectura (PDF, PNG y SVG).
- Documentación de la API (PDF exportado de Swagger y archivo Markdown).
- Colección de pruebas (Postman, Bruno o equivalente).
- Video demostrativo de 5–10 minutos (*[Completar: URL del video demostrativo (5–10 min)]*).

1.2. Modalidades de implementación

Cada equipo implementa el microservicio según lo aprobado en su propuesta:

1. **Modelo local:** entrenamiento y serialización del artefacto (`.pkl`, `.joblib`, `.h5`, etc.) cargado en el servicio.
2. **Modelo mediante API:** el servicio delega inferencia a un proveedor externo; debe documentarse latencia, costo y manejo de errores.

Modalidad declarada por el equipo: *[Completar: Modelo local / Modelo mediante API]*.

Reutilización en el proyecto final

Este componente será **integrado** en la versión completa de la aplicación móvil al cierre del curso. Mantenga contratos de API estables, versionado semántico y trazabilidad de modelos.

2. Objetivos

Para qué sirve esta sección

Enuncia qué debe demostrar el microservicio y cómo se vincula con la problemática de la app móvil.

2.1. Objetivo general

Diseñar, implementar y documentar un microservicio *REST* que ejecute un algoritmo de **aprendizaje no supervisado** relacionado con la problemática del proyecto, capaz de recibir datos, inferir patrones o anomalías, persistir resultados y consultar historial.

2.2. Objetivos específicos

1. Implementar validación y preprocesamiento reproducible de los datos de entrada.
2. Entrenar o cargar un modelo no supervisado (*[Completar: k-means, DBSCAN, Isolation Forest, Autoencoder, ...]*) con evidencia en notebook.
3. Exponer endpoints de inferencia e historial con respuestas JSON bien formadas.
4. Persistir cada inferencia en base de datos con metadatos de trazabilidad.
5. Documentar la API con OpenAPI/Swagger y probar con colección automatizada.
6. Explicar en video cómo el componente se integrará posteriormente con la app móvil.

2.3. Criterios de aceptación mínimos

Requisito	Evidencia esperada
Inferencia	Endpoint POST operativo con JSON de entrada/salida
Historial	Endpoint GET con paginación o filtros básicos
Validación	Errores HTTP 422/400 ante datos inválidos
Persistencia	Registro en BD verificable
Documentación	/docs o openapi.yaml actualizado

3. Descripción del problema

Para qué sirve esta sección

Relaciona la problemática de negocio de la app móvil con la necesidad de un modelo **no supervisado**.

3.1. Contexto del proyecto móvil

[Completar: Describa en 1–2 párrafos la problemática que aborda la aplicación móvil del equipo (dominio, usuarios, decisión que se apoya en datos).]

3.2. Necesidad analítica

Explique por qué un algoritmo **no supervisado** aporta valor antes o durante la operación de la app. Ejemplos frecuentes en minería de datos:

- **Segmentación** de usuarios o transacciones (k-means, clustering jerárquico).
- **Detección de anomalías** en consumo, rutas o comportamiento (Isolation Forest, DBSCAN, autoencoder).
- **Reducción de dimensionalidad** para explorar perfiles (PCA profundo, autoencoder).
- **Descubrimiento de patrones** operativos sin etiquetas previas.

3.3. Pregunta analítica

Formule la pregunta que responde el microservicio:

[Completar: ¿Qué estructura oculta, segmento o caso atípico se busca identificar a partir de las variables X_1 , X_2 , ...?]

3.4. Vínculo con la app móvil (integración futura)

Elemento	Integración posterior
Entrada desde la app	<i>[Completar: qué captura el móvil y envía al API]</i>
Salida hacia la app	<i>[Completar: cluster, score de anomalía, recomendación, alerta]</i>
Frecuencia	<i>[Completar: bajo demanda, batch nocturno, streaming]</i>
Algoritmo seleccionado:	<i>[Completar: k-means, DBSCAN, Isolation Forest, Autoencoder, ...].</i>

4. Arquitectura del componente

Para qué sirve esta sección

Muestra cómo fluyen datos, modelo y persistencia. Adjunte la misma figura en `figures/arquitectura/` como PNG y SVG para los entregables gráficos.

4.1. Vista lógica

La figura siguiente resume el componente. Sustituya el diagrama generado en TikZ por su versión definitiva o coloque `arquitectura.png` en `figures/arquitectura/`.

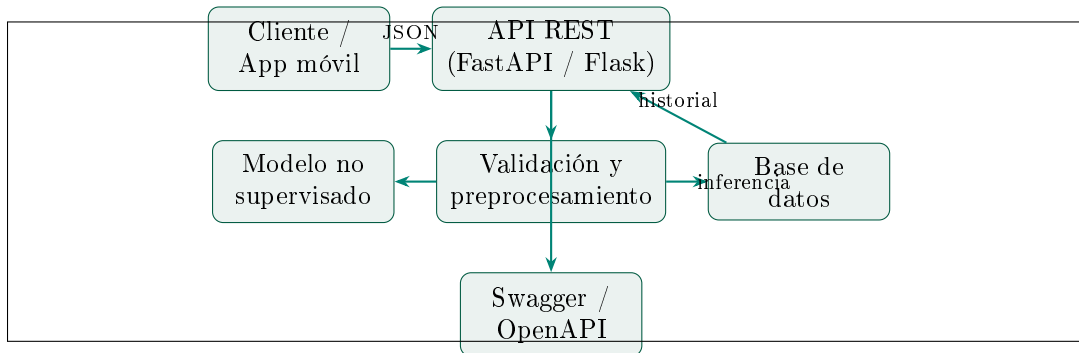


Figura 1: Diagrama lógico del microservicio de ML no supervisado

4.2. Stack tecnológico

Capa	Tecnología	Justificación
API	<i>[Completar: FastAPI / Flask / Express]</i>	<i>[Completar: motivo de elección]</i>
ML	<i>[Completar: scikit-learn / TensorFlow / API externa]</i>	<i>[Completar:]</i>
Serialización	<i>[Completar: joblib / pickle / ONNX]</i>	<i>[Completar:]</i>
Base de datos	<i>[Completar: SQLite / PostgreSQL / MongoDB]</i>	<i>[Completar:]</i>
Contenedor	<i>[Completar: Docker opcional]</i>	<i>[Completar:]</i>

4.3. Flujo de una inferencia

1. Cliente envía petición POST con cuerpo JSON.
2. Capa de validación verifica esquema, tipos y rangos.
3. Preprocesamiento aplica el mismo pipeline del entrenamiento.
4. Modelo calcula cluster, score o representación latente.
5. Servicio persiste entrada, salida, timestamp e identificador.
6. Respuesta JSON al cliente con resultado e `inference_id`.

4.4. Integración futura con la app móvil

[Completar: Describa el contrato previsto: autenticación, URL base, manejo offline, reintentos y qué pantalla consumirá cada endpoint.]

5. Modelo de aprendizaje no supervisado

Para qué sirve esta sección

Documenta el algoritmo, su fundamentación breve y la evidencia de entrenamiento (notebook + métricas).

5.1. Selección del algoritmo

Algoritmo implementado: *[Completar: k-means, DBSCAN, Isolation Forest, Autoencoder, ...]*.
[Completar: Justifique por qué este algoritmo es adecuado para la problemática y los datos disponibles (forma de clusters, densidad, detección de outliers, etc.).]

5.2. Fundamento breve

[Completar: Redacte 2–3 párrafos o ecuaciones clave del método. Ejemplo para k-means: minimización de la inercia; para DBSCAN: regiones de densidad; para Isolation Forest: aislamiento en árboles aleatorios.]

5.3. Datos de entrenamiento

Cuadro 1: Descripción del dataset de entrenamiento

Variable	Descripción	Tipo	Unidad / rango
<i>[Completar: var_1]</i>	<i>[Completar: descripción]</i>	numérica	<i>[Completar: rango]</i>
<i>[Completar: var_2]</i>	<i>[Completar: descripción]</i>	categorica	<i>[Completar: valores]</i>
<i>[Completar: var_3]</i>	<i>[Completar: descripción]</i>	numérica	<i>[Completar: rango]</i>

Metadato	Valor
Fuente	<i>[Completar: archivo, API, scraping ético, etc.]</i>
Registros	<i>[Completar: N]</i>
Periodo	<i>[Completar: fechas]</i>
Licencia / uso	<i>[Completar: académico, interno, público]</i>

5.4. Hiperparámetros

Cuadro 2: Hiperparámetros del modelo no supervisado

Parámetro	Valor	Notas
Algoritmo	<i>[Completar: k-means, DBSCAN, Isolation Forest, Autoencoder, ...]</i>	<i>[Completar: nombre clase sklearn]</i>
<i>[Completar: param_1]</i>	<i>[Completar: valor]</i>	<i>[Completar: ej. n_clusters, gamma]</i>
<i>[Completar: param_2]</i>	<i>[Completar: valor]</i>	<i>[Completar:]</i>
random_state	<i>[Completar: 42]</i>	Reproducibilidad
Preprocesamiento	StandardScaler	<i>[Completar: ajustar si aplica]</i>

5.5. Entrenamiento y artefacto

- Notebook de evidencia: *[Completar: nombre del archivo .ipynb en el repositorio]*.
- Semilla aleatoria: *[Completar: valor documentado]*.
- Partición: *[Completar: train/validación si aplica; muchos modelos no supervisados usan todo el conjunto]*.
- Artefacto generado: *[Completar: ruta del modelo serializado]*.
- Versión del modelo: *[Completar: v1.0.0 o hash del commit]*.

5.6. Métricas de evaluación (no supervisado)

Indique métricas coherentes con el algoritmo:

- Clustering: silhouette, Davies–Bouldin, inercia, estabilidad entre semillas.
- Anomalías: distribución de scores, tasa de alertas en hold-out sintético.
- Autoencoder: error de reconstrucción medio y percentiles para umbral.

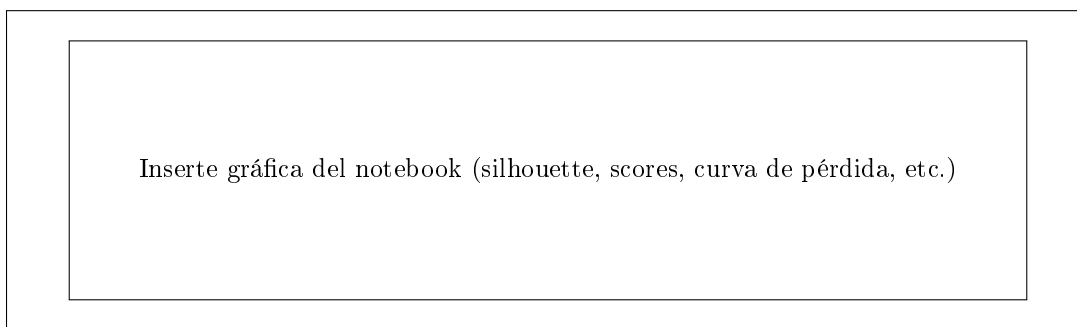


Figura 2: Métricas o gráficos del entrenamiento

6. Preprocesamiento y validación de entrada

Para qué sirve esta sección

El mismo pipeline debe ejecutarse en entrenamiento e inferencia. Documente reglas, escalado y rechazo de datos inválidos.

6.1. Esquema de entrada

[Completar: Describa el JSON de entrada esperado por el endpoint de inferencia (campos, tipos, unidades).]

Ejemplo genérico:

```
{
  "records": [
    {"feature_a": 12.5, "feature_b": 0.03, "feature_c": "categoria_x"}
  ],
  "options": {"return_score": true}
}
```

6.2. Validaciones implementadas

Regla	Comportamiento
Campos obligatorios	HTTP 422 si falta algún atributo requerido
Tipos de dato	Rechazo de strings donde se espera numérico
Rangos	<i>[Completar: límites mín/máx por variable]</i>
Cardinalidad	<i>[Completar: máximo de registros por petición]</i>
Valores nulos	<i>[Completar: imputación o rechazo explícito]</i>

6.3. Transformaciones

1. **Limpieza:** *[Completar: duplicados, outliers groseros, normalización de texto].*
2. **Codificación:** *[Completar: one-hot, label encoding, escalado StandardScaler/MinMax].*
3. **Alineación con el modelo:** orden fijo de columnas igual al entrenamiento.
4. **Serialización del pipeline:** *[Completar: joblib del Pipeline completo o pasos separados].*

```
1 # Ejemplo: pipeline de preprocesamiento + modelo
2 from sklearn.pipeline import Pipeline
3 from sklearn.preprocessing import StandardScaler
4 from sklearn.cluster import KMeans
5
6 pipeline = Pipeline([
7     ("scaler", StandardScaler()),
8     ("model", KMeans(n_clusters=3, random_state=42)),
9 ])
10 # Sustituir por el pipeline real del equipo
```

6.4. Consistencia entrenamiento vs. producción

[Completar: Explique cómo garantizan que el servicio carga exactamente el mismo preprocesamiento usado en el notebook (mismo archivo, misma versión, prueba unitaria, etc.).]

7. Implementación del microservicio

Para qué sirve esta sección

Resume la estructura del repositorio y los módulos críticos. No pegue todo el código: cite rutas y fragmentos representativos.

7.1. Repositorio

URL: *[Completar: URL del repositorio Git]*.

Ruta	Contenido esperado
src/ o app/	Código del servicio
models/	Artefactos serializados
notebooks/	Entrenamiento y EDA
tests/	Pruebas automatizadas
openapi/	Especificación OpenAPI
collections/	Postman/Bruno
Dockerfile	Opcional
README.md	Instalación y ejecución

7.2. Arranque del servicio

```
# Completar con los comandos reales del equipo
python -m venv .venv
source .venv/bin/activate # Windows: .venv\Scripts\activate
pip install -r requirements.txt
uvicorn app.main:app --reload --host 0.0.0.0 --port 8000
```

7.3. Fragmento representativo

[Completar: Pegue 15–30 líneas del handler de inferencia: carga de modelo, preprocesamiento, predicción y persistencia.]

```
1 # app/routes/inference.py (extracto orientativo)
2 @router.post("/api/v1/infer", response_model=InferenceResponse)
3 async def infer(payload: InferenceRequest, db: Session = Depends(get_db)):
4     records = validate_records(payload.records)
5     X = preprocess(records)
6     labels = model.predict(X)
7     inference_id = save_inference(db, records, labels)
8     return {"inference_id": inference_id, "results": labels.tolist()}
```

7.4. Manejo de errores

- 400/422: entrada inválida con detalle en JSON.
- 500: fallo interno; registrar en logs sin filtrar datos sensibles.
- 503: modelo no cargado o dependencia externa caída (modalidad API).

7.5. Dependencias principales

[Completar: Liste librerías clave de requirements.txt con versiones fijadas.]

8. Documentación de la API REST

Para qué sirve esta sección

Enumera endpoints, contratos JSON y referencia a Swagger/OpenAPI. Exporte `/docs` a PDF y mantenga `openapi.yaml` en el repositorio.

8.1. Base URL y versionado

- Base URL local: *[Completar: `http://localhost:8000`]*
- Prefijo de API: *[Completar: `/api/v1`]*
- OpenAPI: `openapi_swagger/openapi.yaml` y UI en `/docs`.

8.2. Endpoints

Cuadro 3: Endpoints REST del microservicio

Método	Ruta	Descripción
POST	<code>/api/v1/infer</code>	Recibe datos, ejecuta modelo, persiste y devuelve JSON
GET	<code>/api/v1/inferences</code>	Lista historial (paginación <code>limit/offset</code>)
GET	<code>/api/v1/inferences/{id}</code>	Detalle de una inferencia
GET	<code>/health</code>	Estado del servicio y carga del modelo
GET	<code>/docs</code>	Documentación Swagger UI
<i>[Completar: otro] [Completar: /ruta]</i>		<i>[Completar: si el equipo agrega endpoints]</i>

8.3. Ejemplo de inferencia

Request

```
POST /api/v1/infer
Content-Type: application/json

{
  "records": [
    { "monto": 1500.0, "frecuencia": 12, "antiguedad_meses": 8 }
  ]
}
```

Response 200

```
{
  "inference_id": "inf_20260326_001",
  "created_at": "2026-03-26T10:15:00Z",
  "results": [
    { "cluster": 2, "anomaly_score": 0.11 }
  ]
}
```

8.4. Consulta de historial

```
GET /api/v1/inferences?limit=20&offset=0

{
```

```

"total": 145,
"items": [
  {
    "inference_id": "inf_20260326_001",
    "created_at": "2026-03-26T10:15:00Z",
    "summary": {"cluster": 2}
  }
]
}

```

8.5. Documentación Swagger

- Archivo fuente: `openapi_swagger/openapi.yaml`.
- Export PDF: *[Completar: adjuntar captura o export en figures/api/swagger.pdf]*.
- Markdown alternativo: *[Completar: API.md en el repositorio]*.

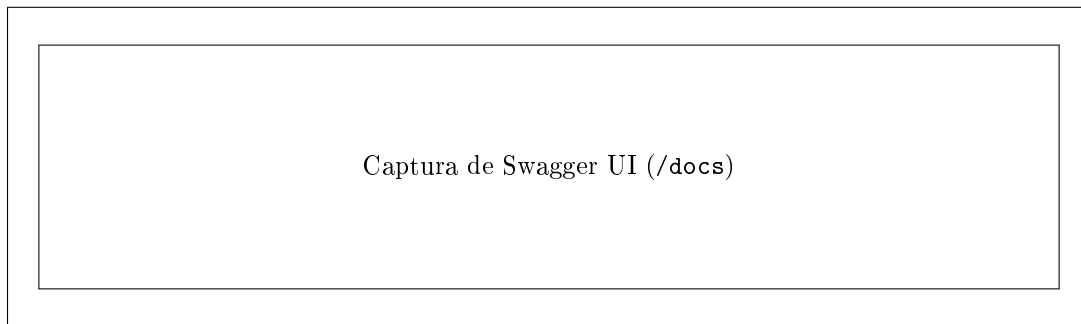


Figura 3: Interfaz Swagger del microservicio

9. Base de datos e historial de inferencias

Para qué sirve esta sección

Cada inferencia debe persistirse para auditoría y consulta posterior. Documente esquema, índices y política de retención.

9.1. Motor y justificación

Motor seleccionado: *[Completar: SQLite / PostgreSQL / MongoDB / otro]*.

[Completar: Explique por qué este motor es suficiente para el hito actual y qué cambiarían al escalar en producción.]

9.2. Modelo de datos

Campo	Tipo	Descripción
id	UUID / serial	Identificador único de inferencia
created_at	timestamp	Fecha UTC de la petición
request_payload	JSON	Entrada validada (sin PII innecesaria)
response_payload	JSON	Salida del modelo
model_version	string	Versión o hash del artefacto
latency_ms	integer	Tiempo de inferencia
client_id	string opcional	Origen (app, postman, batch)

9.3. Operaciones

- **Insert** tras cada inferencia exitosa.
- **Select paginado** para el endpoint de historial.
- **Índices** recomendados: `created_at` DESC, `inference_id`.

9.4. Diagrama entidad-relación (opcional)

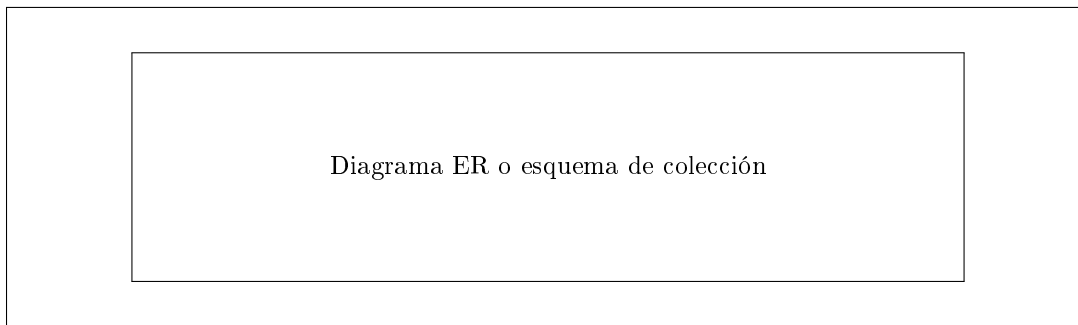


Figura 4: Esquema de persistencia de inferencias

9.5. Privacidad

[Completar: Indique si anonimizan, enmascaran o excluyen campos sensibles antes de guardar.]

10. Pruebas y colección de API

Para qué sirve esta sección

Evidencia de pruebas manuales y automatizadas. Incluya la colección Postman/Bruno en el repositorio y capturas representativas.

10.1. Estrategia de pruebas

Tipo	Qué valida
Unitarias	Preprocesamiento, validadores, funciones puras
Integración	Endpoint + BD + carga de modelo
Contrato	Esquema JSON vs. OpenAPI
Carga ligera	<i>[Completar: opcional: latencia media con N peticiones]</i>

10.2. Colección Postman / Bruno

- Archivo: *[Completar: ruta en repositorio, p. ej. `collections/ml-service.postman_collection.json`]*.
- Variables de entorno: `base_url`, `api_key` si aplica.
- Casos mínimos:
 1. Inferencia válida (200).
 2. Payload incompleto (422).
 3. Tipo incorrecto (422).
 4. Consulta de historial (200).
 5. Historial vacío o paginación.

10.3. Ejecución de pruebas automatizadas

```
# Completar según el stack del equipo
pytest tests/ -v
# o: newman run collections/ml-service.postman_collection.json
```

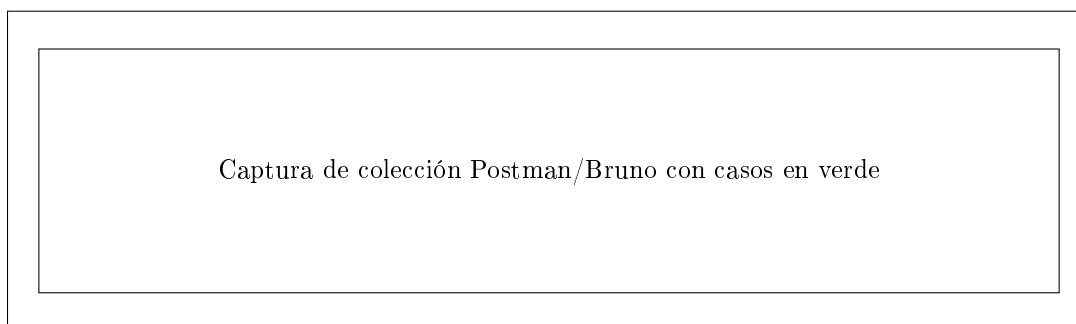


Figura 5: Resultados de la colección de pruebas

10.4. Checklist de entrega

Verificación antes de entregar

- ☐ Inferencia POST probada con datos reales del proyecto.
- ☐ Historial GET devuelve registros persistidos.
- ☐ Swagger accesible y coherente con el código.
- ☐ Notebook reproduce entrenamiento y generación del artefacto.
- ☐ Video de 5–10 min subido y enlazado en portada.

11. Resultados

Para qué sirve esta sección

Presente métricas del modelo, comportamiento del servicio y ejemplos interpretables para la app móvil.

11.1. Resultados del modelo

Cuadro 4: Métricas y resultados del modelo no supervisado

Métrica	Valor	Interpretación
<i>[Completar: silhouette / inercia / etc.]</i>	<i>[Completar: 0.XX]</i>	<i>[Completar: clusters separados]</i>
<i>[Completar: % ruido (DBSCAN)]</i>	<i>[Completar: X %]</i>	<i>[Completar: puntos etiqueta -1]</i>
<i>[Completar: umbral anomalía]</i>	<i>[Completar: score]</i>	<i>[Completar: percentil 95 del train]</i>
Clusters encontrados	<i>[Completar: k]</i>	<i>[Completar: perfiles resumidos]</i>
Estabilidad (semillas)	<i>[Completar: ARI / descripción]</i>	<i>[Completar: si repitieron entrenamiento]</i>

11.2. Ejemplos de inferencia

[Completar: Incluya 3-5 casos reales o sintéticos: entrada, salida del modelo e interpretación para el usuario final de la app.]

```
{
  "input": {"monto": 9800, "frecuencia": 2},
  "output": {"cluster": 0, "label": "perfil_bajo_riesgo"},
  "interpretacion": "Usuario agrupado con baja frecuencia de transacciones."
}
```

11.3. Desempeño del servicio

Métrica	Valor
Latencia media inferencia	<i>[Completar: ms]</i>
Latencia p95	<i>[Completar: ms]</i>
Tamaño artefacto	<i>[Completar: MB]</i>
Peticiones probadas	<i>[Completar: N]</i>

11.4. Visualizaciones

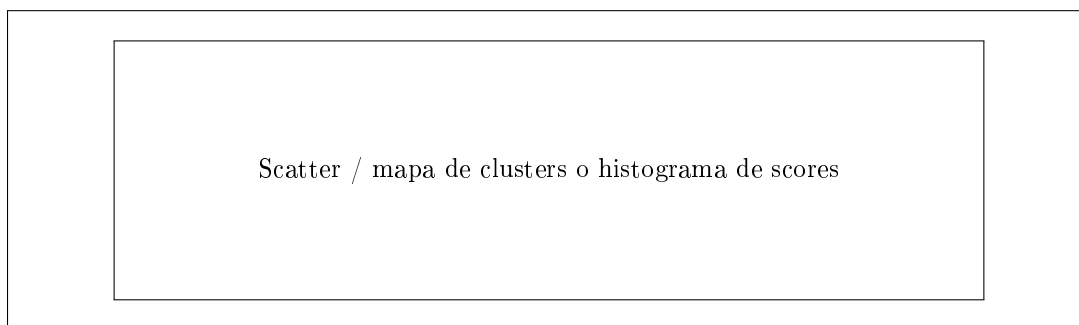


Figura 6: Visualización principal del modelo no supervisado

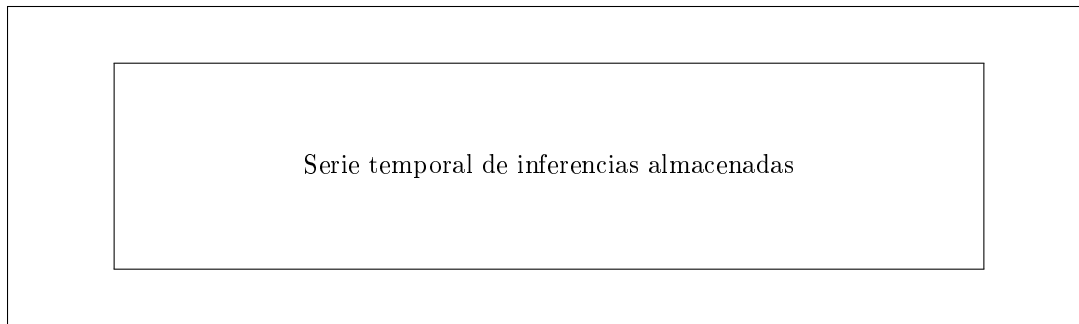


Figura 7: Comportamiento del historial en pruebas

11.5. Limitaciones conocidas

- *[Completar: Datos no representativos, sesgo geográfico, etc.]*
- *[Completar: Sensibilidad a parámetros del algoritmo.]*
- *[Completar: Restricciones de modalidad API externa (cuotas, latencia).]*

12. Conclusiones y trabajo futuro

Para qué sirve esta sección

Cierra el hito: qué funciona hoy, qué falta para integrar con la app móvil y qué mejoras planean.

12.1. Conclusiones

1. *[Completar: Logro principal del microservicio respecto a la problemática del proyecto.]*
2. *[Completar: Grado de cumplimiento de validación, inferencia, persistencia y documentación.]*
3. *[Completar: Lección aprendida sobre no supervisado vs. necesidades del negocio.]*

12.2. Integración con la aplicación móvil

[Completar: Describa en un párrafo el flujo previsto: pantalla de la app → llamada HTTP → interpretación del JSON → acción visible para el usuario.]

12.3. Trabajo futuro

- Autenticación (JWT, API keys) y HTTPS en despliegue.
- Reentrenamiento programado y versionado de modelos (MLOps).
- Monitoreo de *data drift* y recalibración de umbrales.
- Contenedorización y despliegue en nube institucional.
- Pruebas de carga y alta disponibilidad.

12.4. Entregables finales del hito

Entregable	Estado
Repositorio	<i>[Completar: URL + tag/commit]</i>
PDF (L ^A T _E X)	Este documento
Notebook	<i>[Completar: ruta]</i>
Diagrama arquitectura	PDF, PNG, SVG en <code>figures/arquitectura/</code>
API docs	PDF + Markdown
Colección pruebas	Postman/Bruno
Video 5–10 min	<i>[Completar: URL del video demostrativo (5–10 min)]</i>

A. Anexos

A.1. Anexo A — Especificación OpenAPI

Incluya o referencie el archivo `openapi_swagger/openapi.yaml`. En el repositorio mantenga también `openapi_swagger/API.md`. Exporte la UI de Swagger (`/docs`) a PDF para el entregable gráfico.

```
openapi: 3.0.3
info:
  title: Microservicio ML no supervisado
  version: 1.0.0
paths:
  /api/v1/infer:
    post:
      summary: Ejecutar inferencia no supervisada
  /api/v1/inferences:
    get:
      summary: Consultar historial de inferencias
```

Sustituya el fragmento anterior por la especificación completa generada desde su framework (FastAPI exporta OpenAPI automáticamente).

A.2. Anexo B — Enlaces de entrega

Recurso	Enlace / ruta
Repositorio Git	<i>[Completar: URL del repositorio Git]</i>
Video demostrativo	<i>[Completar: URL del video demostrativo (5–10 min)]</i>
Notebook	<i>[Completar: ruta en repo]</i>
Colección Postman/Bruno	<i>[Completar: ruta en repo]</i>
API.md	<i>[Completar: ruta en repo]</i>

A.3. Anexo C — Guion sugerido del video (5–10 min)

1. Contexto del proyecto móvil y problemática (1 min).
2. Arquitectura del microservicio (1–2 min).
3. Entrenamiento del modelo no supervisado en notebook (2 min).
4. Demo en vivo: inferencia + consulta de historial (2–3 min).
5. Swagger y colección de pruebas (1 min).

6. Cómo se integrará con la app móvil al final del curso (1 min).

A.4. Anexo D — Registro de decisiones técnicas (opcional)

[Completar: Tabla breve: fecha, decisión, alternativa descartada, motivo.]

Referencias

- [1] J. Han, M. Kamber, and J. Pei, *Data Mining: Concepts and Techniques*. Morgan Kaufmann, 3rd ed., 2011.
- [2] F. Pedregosa *et al.*, “Scikit-learn: Machine learning in python.” <https://scikit-learn.org>, 2011.
- [3] “Openapi specification.” <https://swagger.io/specification/>, 2024.